

PROJET PERSONEL

Rapport de projet

Système de recommandation de films

Filtrage collaboratif sur les notes SensCritique

Auteur : Alexandre Irazoqui

Année : 2025–2026

Table des matières

1	Introduction	2
2	Constitution du jeu de données (scraping)	2
2.1	Choix de la source	2
2.2	Choix du corpus de films	2
2.3	Considérations éthiques	3
2.4	Exploration et filtrage	3
3	Le modèle	4
3.1	Factorisation de matrices et choix de l'ALS	4
3.2	Validation et réglage des hyperparamètres	4
3.3	Biais de popularité et correction IPS	5
3.3.1	Diagnostic	5
3.3.2	Correction par Inverse Propensity Scoring	6
3.3.3	Évaluation de la correction	7
3.3.4	Validation qualitative	8
4	Déploiement de la maquette	8
4.1	Choix techniques	8
4.2	Affiches et données personnelles	9
4.3	Le problème du démarrage à froid	9
5	Conclusion	10
6	Sources	10

1 Introduction

L'objectif de ce projet est de construire un **modèle de recommandation de films** exploitant le *feedback explicite* des utilisateurs, c'est-à-dire les notes qu'ils attribuent aux films, par opposition au feedback implicite (clics, temps de visionnage).

Le projet se divise en trois parties :

1. le **scraping**, c'est-à-dire la constitution du jeu de données ;
2. la **création du modèle**, une factorisation de matrices entraînée par moindres carrés alternés (ALS), puis débiaisée en popularité ;
3. le **déploiement** d'une maquette web interactive.

L'ambition est d'obtenir un modèle « cinéphile » : un système qui prenne réellement en compte les goûts de l'utilisateur, sans se contenter de lui faire des recommandations « évidentes » (les mêmes blockbusters pour tout le monde). Cette exigence guidera les choix techniques tout au long du rapport, en particulier la correction du biais de popularité (section 3.3).

Dans ce rapport, nous distinguons systématiquement la **partie méthodologique** (les choix et leur justification) de la **partie implémentation** (la logique effectivement retenue dans le code).

2 Constitution du jeu de données (scraping)

2.1 Choix de la source

Nous avons besoin d'un site où les utilisateurs notent leurs films. Notre choix s'est porté sur **SensCritique**, en raison de sa forte communauté francophone et de la richesse de ses données publiques de notation.

Techniquement, la collecte passe par l'API GraphQL de la plateforme (apollo.senscritique.com). Deux scripts se partagent le travail :

- `scraper/collection.py` récupère la liste des identifiants de films d'une collection utilisateur (requête `UserCollection`, paginée) ;
- `scraper/scrape_all.py` itère sur ces identifiants et collecte toutes les notes de chaque film via la requête `ProductUserInfos` (pages de 40 notes, délai de 1 s entre les requêtes).

Le scraping est **interruptible et reprenable** : les identifiants déjà traités sont consignés dans `done_ids.json`, et les notes sont écrites en mode *append* dans un CSV. Chaque requête est en outre réessayée jusqu'à trois fois en cas d'échec réseau.

Une passe de complétion a ensuite été nécessaire : une soixantaine de films notoires (*2001 : l'Odyssée de l'espace*, *Citizen Kane*, la filmographie de Pasolini...) manquaient au corpus à cause d'échecs de scraping (les films les plus notés sont les plus longs à paginer, donc les plus exposés aux timeouts) et ont été re-scrapés, afin d'éviter qu'un utilisateur de la maquette ne tombe sur un « film introuvable » pour des titres aussi connus.

2.2 Choix du corpus de films

Plutôt que de partir des classements « Top films » de la plateforme, nous avons pris comme base les films vus par **Moizi**, un utilisateur réputé de SensCritique ayant noté environ 5 000 films. Ce choix est délibéré : partir des tops de « meilleurs films » aurait introduit un biais important,

- en sur-représentant les films acclamés ou populaires ;
- en excluant systématiquement les films de niche, anciens ou peu notés ;
- en produisant une matrice qui reflète le goût pour les *bons* films, et non le goût en général.

La collection large et éclectique d'un seul cinéphile fournit au contraire un corpus **divers et représentatif**, tout en restant de taille raisonnable (~ 5 300 films).

2.3 Considérations éthiques

- Toutes les données collectées sont **publiques** sur SensCritique (les notes sont publiques par défaut, sans authentification).
- Le `robots.txt` de la plateforme ne restreint pas l'accès aux pages visées par le scraper.
- Aucune donnée personnelle n'est collectée au-delà d'identifiants utilisateurs anonymes et de notes numériques.
- Le scraper applique un **rate limiting** (délais entre requêtes) pour ne pas surcharger les serveurs.
- J'ai contacté SensCritique par e-mail pour demander une autorisation explicite d'utilisation à des fins de recherche non commerciale (sans réponse à ce jour).
- Le jeu de données n'est **pas redistribué** : seul le code de scraping et de modélisation est partagé.

2.4 Exploration et filtrage

L'exploration du jeu de données brut est documentée dans le notebook `notebook/Films.ipynb`. Les chiffres clés sont les suivants :

Métrique	Valeur
Notes brutes	$\sim 47,5$ millions
Utilisateurs uniques	$\sim 582\,000$
Films uniques	5 336
Sparsité brute	$\sim 98,5\%$

TABLE 1 – Jeu de données brut après scraping.

L'analyse de la distribution des notes, du nombre de notes par utilisateur et par film (histogrammes en échelle logarithmique) a guidé le choix des seuils de filtrage. Le tableau 2 montre l'effet de différents seuils sur le nombre minimal de notes par utilisateur :

Seuil	Users gardés	% users	Notes gardées	% notes
10	255 214	43,8 %	46 686 633	98,2 %
20	207 157	35,6 %	46 026 933	96,8 %
50	153 469	26,4 %	44 305 374	93,2 %
100	114 315	19,6 %	41 489 031	87,2 %

TABLE 2 – Impact du seuil minimal de notes par utilisateur.

Le seuil de **20 notes minimum par utilisateur** s'impose : il élimine près de deux tiers des utilisateurs (64,4 % : les profils trop creux, peu informatifs) tout en ne perdant que 3,2 % des notes, un excellent compromis entre la masse de données et la perte d'information. Nous filtrons également les films ayant moins de **50 notes**, ce qui ne retire qu'environ 250 films de la longue traîne (la médiane est à $\sim 3\,400$ notes par film), et nous retirons l'utilisateur *seed* (Moizi) dont la collection a servi de base, pour ne pas biaiser le modèle vers son profil (d'où 207 156 utilisateurs au final, contre 207 157 au seuil).

Après filtrage :

Les identifiants sont ensuite encodés en indices contigus (`LabelEncoder`), et la matrice de notes $R \in \mathbb{R}^{207\,156 \times 5\,086}$ est construite au format creux `scipy.sparse CSR`.

Métrique	Valeur
Notes filtrées	~ 46 millions
Utilisateurs uniques	207 156
Films uniques	5 086
Sparsité	$\sim 95,6$ %

TABLE 3 – Jeu de données après filtrage.

3 Le modèle

3.1 Factorisation de matrices et choix de l’ALS

La matrice R étant très creuse, l’approche qui s’impose est la **factorisation de matrices** popularisée lors du prix Netflix [1]. On cherche à approcher chaque note observée r_{ui} par

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i, \quad (1)$$

où μ est la moyenne globale des notes, b_u et b_i les **biais** utilisateur et film (un utilisateur sévère, un film consensuel...), et $p_u, q_i \in \mathbb{R}^k$ les **facteurs latents** de l’utilisateur u et du film i . L’usage des biais est indispensable : une grande partie de la variance des notes s’explique par ces effets de niveau, indépendamment de toute interaction goût/film [1].

On minimise l’erreur quadratique régularisée sur les notes *observées* ($\mathcal{K}$ désigne l’ensemble des couples (u, i) observés) :

$$\min_{P, Q} \sum_{(u, i) \in \mathcal{K}} (r_{ui} - \mu - b_u - b_i - p_u^\top q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2). \quad (2)$$

Pour optimiser (2), deux grandes familles existent : la descente de gradient stochastique (SGD) et les **moindres carrés alternés** (*Alternating Least Squares*, ALS). Nous avons retenu l’ALS, implémenté *from scratch* en NumPy (`models/ALS.py`) : le problème (2) n’est pas convexe conjointement en (P, Q) , mais il *devient* un simple problème de moindres carrés dès que l’on fixe l’un des deux blocs. À Q fixé, chaque vecteur utilisateur admet une solution analytique :

$$p_u = (Y_u^\top Y_u + \lambda I)^{-1} Y_u^\top \tilde{r}_u, \quad (3)$$

où Y_u empile les facteurs des films notés par u et $\tilde{r}_u = r_u - \mu - b_u - b_i$ sont les notes recentrées ; et symétriquement pour les films à P fixé. On alterne ces deux mises à jour jusqu’à convergence (15 itérations en pratique, le RMSE étant suivi sur un échantillon fixe de 200 000 notes).

Un argument classique en faveur de l’ALS est que chaque mise à jour (3) est indépendante des autres lignes, donc **parallélisable** [1]. En pratique, nous avons testé cette parallélisation mais ne l’avons *pas* retenue : sur nos systèmes linéaires de petite taille ($k \times k$ avec $k = 20$), le gain était minime : nous forçons même BLAS en mono-thread (`threadpoolctl`), le multi-threading n’apportant rien sur des résolutions 20×20 .

Implémentation des biais. Les biais μ , b_u , b_i sont précalculés une fois avant l’alternance (baseline de biais, puis factorisation du résidu). L’ALS biaisé classique les réapprend à chaque itération ; notre variante est plus simple et donne des résultats très proches en pratique.

3.2 Validation et réglage des hyperparamètres

Sanity check. Avant d’entraîner notre ALS, nous avons validé le pipeline avec la bibliothèque **surprise** (SVD entraînée par SGD) sur un échantillon de 500 000 notes. Cette baseline donne un **RMSE de 1,47** et une **MAE de 1,12** (échelle 1–10), ce qui fixe un ordre de grandeur de référence.

Réglage. Les hyperparamètres k et λ ont été choisis par grid search (`models/grid_search_als.py`) sur une découpe train/validation/test 64/16/20 des notes : la grille $k \in \{10, 20, 50\} \times \lambda \in \{5, 20, 50\}$ est évaluée au RMSE du jeu de validation, le test n’étant utilisé qu’une seule fois, pour les chiffres finaux. La configuration retenue est :

$$k = 20 \text{ facteurs}, \quad \lambda = 20, \quad 15 \text{ itérations},$$

le nombre d’itérations étant fixé par convergence du RMSE (suivi sur un échantillon fixe) et non par la grille.

Une régularisation forte est nécessaire : avec 46 M de notes mais une matrice encore creuse à 95,6 %, des valeurs faibles de λ font diverger les facteurs des films peu notés.

L’exposant IPS α n’est en revanche pas réglé au RMSE : conformément à l’argument développé en section 3.3 (le RMSE global, dominé par les films populaires, est un mauvais juge du débiaisage), il est sélectionné sur la métrique cible, la réduction du biais de popularité des recommandations, à précision de prédiction neutre.

Résultats de prédiction. Sur le jeu de test :

Métrique	Valeur
RMSE train	1,14
RMSE test	1,24

TABLE 4 – Erreur de prédiction de notes (échelle 1–10).

Résultats de classement. La prédiction de note n’est qu’un proxy : ce qui compte pour un recommandeur, c’est le *classement*. Deux protocoles sont évalués :

- **Protocole A** : tous les positifs test de l’utilisateur + 100 négatifs tirés au hasard ;
- **Protocole B** (*leave-one-out* à la NeuMF [3]) : le film préféré de l’utilisateur dans le test + 99 négatifs.

Protocole	Métrique	Valeur
A – positifs test + 100 négatifs	NDCG@10	0,42
A – positifs test + 100 négatifs	MAP@10 (note ≥ 7)	0,31
B – 1 positif + 99 négatifs	HR@10	0,55
B – 1 positif + 99 négatifs	NDCG@10	0,36

TABLE 5 – Métriques de classement sur le jeu de test.

Avec un positif parmi 100 candidats, un classement aléatoire donnerait $\text{HR@10} \approx 0,10$: le **0,55** du modèle montre qu’il capte un vrai signal de goût.

3.3 Biais de popularité et correction IPS

3.3.1 Diagnostic

En examinant les recommandations produites, nous avons mesuré un fort **biais de popularité** : la popularité moyenne des films recommandés en top-10 est de 32 514 notes, contre 9 048 pour l’ensemble du catalogue, soit un ratio de **3,59**×

Ce biais est structurel : les notes ne sont pas manquantes au hasard (*Missing Not At Random*) : on note les films que l’on a vus, et l’on voit surtout des films populaires. Un modèle entraîné naïvement sur ces observations sur-apprend donc les blockbusters, ce qui est contraire à notre objectif de modèle « cinéphile ».

Quantile	Catalogue global	Top recommandations
25 %	838	16 372
50 %	3 372	29 325
75 %	10 848	46 087
90 %	24 534	62 087

TABLE 6 – Popularité (nombre de notes) : catalogue vs films recommandés.

3.3.2 Correction par Inverse Propensity Scoring

Pour corriger ce biais, nous repondérons la fonction de perte par **Inverse Propensity Scoring** (IPS), en suivant le cadre proposé par Schnabel et al. [2] : si chaque observation est repondérée par l'inverse de sa probabilité d'être observée (sa *propension*), alors l'estimateur du risque devient sans biais vis-à-vis de la distribution complète des couples (utilisateur, film).

Nous modélisons la propension **au niveau de l'item** : pour un film i noté par n_i utilisateurs sur N ,

$$p_i = \frac{n_i}{N}, \quad w_i = \left(\frac{1}{p_i}\right)^\alpha, \quad \text{puis } w_i \leftarrow \frac{w_i}{w}. \quad (4)$$

Un film populaire (p_i grand) est sur-représenté dans les observations : on le *sous*-pondère ; un film de niche est rare : on le *sur*-pondère. C'est précisément ce rééquilibrage qui débiaise la popularité. Deux précisions :

- l'exposant $\alpha \leq 1$ **tempère les poids extrêmes** (un ratio de propension de $1000\times$ devient $\sim 30\times$ pour $\alpha = 0,5$ et $\sim 8\times$ pour $\alpha = 0,3$), ce qui contrôle le coût en variance de l'IPS : les films de niche, très sur-pondérés, reçoivent de fait moins de régularisation effective et risqueraient l'overfitting avec des poids pleins ;
- la normalisation à moyenne 1 est un simple rescale qui ne change pas l'équilibre relatif entre films, mais garde le terme d'erreur à la même échelle que la version non pondérée, λ reste donc comparable entre les deux modèles.

Il s'agit bien d'une correction du biais de *popularité* (qui note quoi), et non de la correction de *positivité* de Schnabel et al. : on ne touche pas à la valeur des notes.

La perte devient

$$\min_{P, Q} \sum_{(u,i) \in \mathcal{K}} w_i (r_{ui} - \mu - b_u - b_i - p_u^\top q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2), \quad (5)$$

et les mises à jour ALS deviennent des **moindres carrés pondérés** :

- **côté utilisateur**, c'est là que la correction agit réellement :

$$p_u = (Y_u^\top W_u Y_u + \lambda I)^{-1} Y_u^\top W_u \tilde{r}_u, \quad (6)$$

avec $W_u = \text{diag}(w_i)$ sur les films notés par u . Concrètement, le goût de u est ajusté surtout sur ses films de niche (w_i grand) et moins sur les blockbusters (w_i petit) ;

- **côté item**, le poids w_i est constant pour tous les utilisateurs d'un même film, donc il se factorise :

$$q_i = (w_i X_i^\top X_i + \lambda I)^{-1} w_i X_i^\top \tilde{r}_i. \quad (7)$$

L'effet net porte sur l'équilibre données/régularisation : un blockbuster (w_i petit) est régularisé plus fortement qu'un film de niche (w_i grand).

Avec $w_i = 1$ partout, on retombe exactement sur l'ALS classique : la comparaison pondéré / non pondéré se fait donc à code identique, en basculant un simple drapeau `use_ips`.

3.3.3 Évaluation de la correction

L'évaluation de l'IPS demande de la prudence : le RMSE global du test est *dominé par les films populaires*, que l'on vient justement de sous-pondérer. Un RMSE global légèrement dégradé ne signifie donc pas que la correction échoue. Le protocole de `models/bench_ips.py` compare ALS baseline et ALS+IPS ($\alpha \in \{1,0; 0,5; 0,3\}$) sur :

- le **RMSE découpé par tertile de popularité** (niche / moyen / populaire), avec une baseline « biais seuls » ;
- le **classement en protocole B avec négatifs appariés en popularité** (bande logarithmique $\pm 20\%$ autour du positif), avec des baselines « popularité pure » et « biais seuls ». L'appariement neutralise l'avantage trivial qu'un modèle popularité-pure aurait sur des négatifs tirés au hasard ;
- le **biais de popularité des recommandations** lui-même (le ratio top-10 / catalogue du diagnostic ci-dessus, recalculé avec le modèle débiaisé), c'est la métrique cible du projet.

Résultats. Les tableaux 7 et 8 donnent les résultats complets (split train/test : 36,8 M / 9,2 M de notes).

Tertile (popularité)	Biais seuls	Baseline	IPS $\alpha=1,0$	IPS $\alpha=0,5$	IPS $\alpha=0,3$
Niche ($\leq 12\,189$)	1,4089	1,2649	1,3379	1,2647	1,2597
Moyen ($12\,189\text{--}29\,887$)	1,4152	1,2683	1,3682	1,2633	1,2593
Populaire ($> 29\,887$)	1,3659	1,2264	1,3349	1,2142	1,2072
Global	1,3971	1,2535	1,3473	1,2478	1,2425

TABLE 7 – RMSE de test par tertile de popularité (gras = meilleur).

Modèle	HR@10	NDCG@10
Popularité pure	0,3280	0,2236
Biais seuls	0,4130	0,2202
Baseline (ALS)	0,4790	0,3004
IPS $\alpha = 1,0$	0,4220	0,2300
IPS $\alpha = 0,5$	0,4660	0,2758
IPS $\alpha = 0,3$	0,4780	0,2950

TABLE 8 – Classement, protocole B avec négatifs appariés en popularité (repère hasard : HR@10 $\approx 0,10$).

Effet sur le biais de popularité. C'est le résultat central du projet : avec le modèle IPS $\alpha = 0,3$, la popularité moyenne des films recommandés en top-10 passe de 32 514 à 24 627 notes, soit un ratio qui descend de **3,59 $\times$** à **2,72 $\times$** : une réduction d'environ **24 %** du biais de popularité des recommandations.

Analyse. Quatre enseignements :

- **L'IPS plein ($\alpha = 1,0$) paie cher son débiaisage** : le RMSE se dégrade sur tous les tertiles (1,3473 en global contre 1,2535 pour la baseline) et le classement chute (HR@10 : 0,4220 contre 0,4790). C'est le coût en variance annoncé par la théorie [2] : les poids extrêmes sous-régularisent les films de niche.
- **Le tempérament des poids récupère la précision ; mais pas plus.** À première lecture, le tableau 7 suggère que l'IPS $\alpha = 0,3$ fait *mieux* que la baseline en RMSE (1,2425

contre 1,2535). J'avais d'abord soupçonné que ce léger gain de RMSE était un artefact de configuration ; le re-run complet du benchmark, à configuration strictement identique, l'a reproduit. La conclusion prudente reste néanmoins la **neutralité** : l'écart est faible, et l'essentiel est que le débiaisage ne coûte rien en précision de prédiction.

- **Le vrai gain est sur la métrique cible** : -24% de biais de popularité des recommandations ($3,59\times \rightarrow 2,72\times$) pour un coût de précision nul. C'est exactement le contrat de l'IPS tempéré, et c'est la métrique qui correspond à l'objectif « cinéphile » du projet, pas le RMSE.
- **La factorisation apporte un vrai signal** : tous les modèles MF dominent largement les baselines popularité pure et biais seuls sur les deux métriques, alors même que l'appariement en popularité des négatifs neutralise l'avantage trivial de la popularité.

Une limite reste documentée : pour les profils *exclusivement* mainstream, l'IPS sous-pondère précisément les films qui définissent leur goût et tend à sur-corriger leurs recommandations : effet quantifiable par la norme plus faible de leur vecteur latent. Le tempérament $\alpha = 0,3$ minimise cette sur-correction sans l'annuler ; c'est une propriété assumée du débiaisage, pas un bug.

Nous retenons donc $\alpha = 0,3$ pour le modèle final. Deux leçons méthodologiques méritent d'être soulignées. D'abord, le RMSE global, dominé par les films populaires, est un **indicateur trompeur** pour juger une correction de biais : il fallait le découper par popularité, et surtout mesurer directement la métrique cible (le biais des recommandations) plutôt que la seule précision. La précision de prédiction et la pertinence des recommandations sont deux objectifs distincts, et le second est celui qui nous importe.

3.3.4 Validation qualitative

L'espace latent appris est interprétable. Les plus proches voisins (cosinus sur les facteurs Q) de quelques films :

Film requête	Plus proches voisins (espace latent)
Stalker	Solaris (0,95), Le Miroir (0,95), Andreï Roublev (0,93), Huit et demi (0,92)
Le Voyage de Chihiro	Princesse Mononoké (0,95), Le Château ambulant (0,89), Mon voisin Totoro (0,87)
Shining	Full Metal Jacket (0,84), Orange mécanique (0,82), Taxi Driver (0,81), Psychose (0,81)
The Dark Knight	The Dark Knight Rises (0,79), Interstellar (0,74), Casino Royale (0,73)

TABLE 9 – Voisinages dans l'espace latent : le modèle capte des proximités d'auteur (Tarkovski), de studio (Ghibli) et de genre, sans aucune métadonnée de contenu.

Le modèle retrouve la filmographie de Tarkovski autour de *Stalker* et l'univers Ghibli autour de *Chihiro* **sans jamais avoir vu ni réalisateur, ni studio, ni genre** : uniquement par corrélation des notes.

4 Déploiement de la maquette

4.1 Choix techniques

La maquette est une application **Streamlit** déployée sur **HuggingFace Spaces**. Streamlit permet de produire très rapidement une interface interactive en Python pur (exactement ce qu'il faut pour une maquette), et HuggingFace Spaces offre un hébergement gratuit adapté aux démonstrations de modèles.

Le modèle complet (facteurs utilisateurs P et matrice creuse d'entraînement comprises) est trop lourd pour ce cadre. Nous générons donc un **cache d'inférence allégé** (~ 21 Mo) qui ne

conserve que ce dont le fold-in a besoin : les facteurs films Q (en `float32`), les biais μ et b_i , les poids IPS w_i , la régularisation λ et la table des titres. L'inférence pour un nouvel utilisateur se fait alors **en temps réel, sans réentraînement**, par résolution analytique de son vecteur latent.

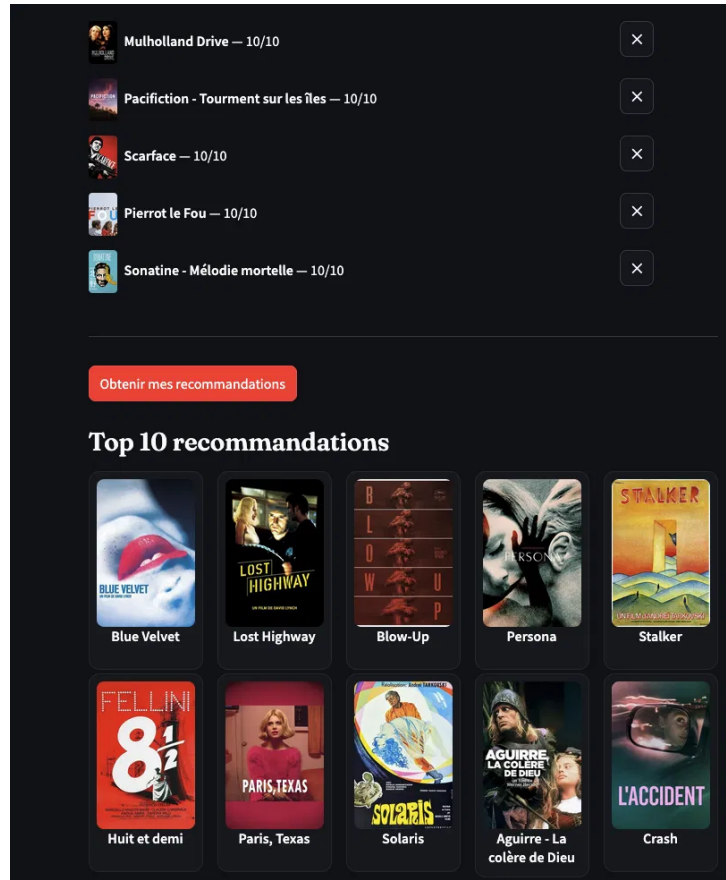


FIGURE 1 – La maquette déployée : films notés par l'utilisateur et top 10 des recommandations.

4.2 Affiches et données personnelles

Les affiches de films sont récupérées via l'**API TMDb**. Dans une première version, nous interrogeons directement SensCritique pour les affiches *et* pour importer les films vus par un utilisateur à partir de son pseudo. Nous avons retiré ces deux fonctionnalités par **prudence vis-à-vis du droit des données** : la maquette publique ne contacte plus SensCritique du tout. De même, le jeu de données n'est pas publié, et le modèle déployé sur HuggingFace l'est avec un cache vidé de toute donnée utilisateur.

L'import de notes se fait désormais par fichier CSV : export natif **Letterboxd**, ou export SensCritique via l'outil tiers Sensboxd. Les titres sont rattachés aux identifiants SensCritique par une table de correspondance construite sur les titres originaux, complétée par une recherche floue.

4.3 Le problème du démarrage à froid

Le point dur du déploiement a été le **démarrage à froid** (*cold start*) : produire de bonnes recommandations pour un utilisateur inconnu du modèle. L'approche standard est le **fold-in ALS** : on résout analytiquement le vecteur latent du nouvel utilisateur à partir de ses notes,

$$p_{\text{new}} = (Y^{\top} W Y + \lambda I)^{-1} Y^{\top} W \tilde{r}, \quad (8)$$

exactement comme une mise à jour utilisateur de l'entraînement.

En pratique, le fold-in s’est révélé difficile à faire fonctionner avec peu de notes : **la variance du vecteur résolu n’est pas assez grande**, et l’on se retrouve face à un dilemme : selon le réglage, le modèle sur-apprend les quelques notes fournies quand il y en a beaucoup et pas quand il y en a peu, ou l’inverse. Aucun réglage unique ne convenait aux deux régimes.

Nous avons donc fait un **compromis à seuil** :

- **moins de 20 notes** : score par **similarité cosinus au centroïde** des films notés, pondéré par les notes (poids $\max(r - \mu, 0, 1)$). Le fold-in ALS n’a pas assez de signal dans ce régime avec le modèle IPS ;
- **20 notes ou plus** : véritable **fold-in ALS**, avec scores de prédiction /10 affichés.

Ce seuil épouse les usages réels : soit l’utilisateur saisit ses films à la main et il n’en entrera jamais plus de 20, soit il importe son profil complet (Letterboxd) et il y en aura bien plus de 20. Notons enfin le rôle des poids IPS au fold-in : ils interviennent dans la *résolution* du vecteur (W dans l’équation ci-dessus), exactement comme à l’entraînement, leur omission ferait s’effondrer p_{new} vers le canon, les films populaires (nombreux dans tout profil) dominant le solve par leur nombre. Ils n’interviennent en revanche pas dans le *score* final $q_i^\top p_{\text{new}}$: la correction IPS agit sur l’estimation du profil, pas sur le classement affiché.

5 Conclusion

Ce projet couvre la chaîne complète d’un système de recommandation : la **constitution d’un jeu de données** original de 46 millions de notes par scraping raisonné et éthique de SensCritique ; l’**implémentation from scratch** d’une factorisation de matrices par ALS avec biais [1], validée contre une baseline SVD ; le **diagnostic puis la réduction mesurée** d’un biais de popularité des recommandations ($3,59\times \rightarrow 2,72\times$, soit -24%) par repondération IPS [2] ; et le **déploiement** d’une maquette web en inférence temps réel par fold-in.

Les deux enseignements principaux que j’en ai tiré sont méthodologiques. D’abord, les données d’observation ne sont pas neutres : le caractère *Missing Not At Random* des notes impose de débiaiser, sous peine de construire un recommandeur de blockbusters. Ensuite, la métrique globale n’est pas l’objectif final : le RMSE global aurait masqué l’essentiel. C’est en le découpant par popularité, en appariant les négatifs d’évaluation, et en mesurant directement le biais des recommandations que l’on a pu montrer que l’IPS tempéré ($\alpha = 0,3$) débiaise à précision *strictement neutre*.

6 Sources

Références

- [1] Y. Koren, R. Bell, C. Volinsky. *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer, vol. 42, n°8, 2009.
- [2] T. Schnabel, A. Swaminathan, A. Singh, N. Chandak, T. Joachims. *Recommendations as Treatments : Debiasing Learning and Evaluation*. Proceedings of ICML, 2016.
- [3] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, T.-S. Chua. *Neural Collaborative Filtering*. Proceedings of WWW, 2017. (Protocole d’évaluation *leave-one-out*.)
- [4] SensCritique — <https://www.senscritique.com> (API GraphQL [apollo.senscritique.com](https://www.senscritique.com/apollo)).
- [5] The Movie Database (TMDB) — <https://www.themoviedb.org> (API affiches).
- [6] N. Hug. *Surprise : A Python library for recommender systems*. Journal of Open Source Software, 2020. (Baseline SVD.)
- [7] Maquette déployée : <https://huggingface.co/spaces/AlexandreIrazoqui/MovieRecommender>.